Introduction to SQL and Advanced Functions | Assignment

Question 1 : Explain the fundamental differences between DDL, DML, and DQL commands in SQL. Provide one example for each type of command

Answer- In SQL, commands are broadly classified into DDL (Data Definition Language), DML (Data Manipulation Language), and DQL (Data Query Language), based on their functionality. DDL commands are used to define and modify the structure of the database. These commands deal with creating, altering, and deleting database objects such as tables and schemas. For example, the CREATE TABLE Students (id INT, name VARCHAR(50), age INT); command is a DDL command that creates a new table named *Students* with specified columns.

On the other hand, DML commands are used to manipulate the data stored inside the database tables. These commands allow users to insert, update, and delete records in a table. For instance, the INSERT INTO Students (id, name, age) VALUES (1, 'Akash', 20); command is a DML operation that adds a new record to the Students table.

DQL commands are used to retrieve data from the database. The primary command in this category is SELECT, which is used to fetch data based on specific conditions. For example, the SELECT * FROM Students; command retrieves all records from the Students table. Unlike DML, DQL does not modify the data but only displays it.

In summary, DDL is used for defining the database structure, DML is used for modifying the data, and DQL is used for querying and retrieving the data from the database.

Question 2 : What is the purpose of SQL constraints? Name and describe three common types of constraints, providing a simple scenario where each would be useful.

Answer- SQL constraints are rules applied to table columns to ensure the accuracy, reliability, and integrity of the data stored in a database. The main purpose of constraints is to restrict the type of data that can be inserted into a table, thereby preventing invalid or inconsistent data entries. Constraints help maintain data quality and enforce business rules automatically at the database level.

One common type of constraint is the PRIMARY KEY constraint. It is used to uniquely identify each record in a table, and it does not allow duplicate or NULL values. For example, in a Students table, the *id* column can be set as a primary key so that each student has a unique ID, ensuring no two students share the same identifier.

Another important constraint is the FOREIGN KEY constraint. It is used to establish a relationship between two tables by linking a column in one table to the primary key of another table. This ensures referential integrity. For instance, in an Orders table, a *student_id* column can be a foreign key that references the *id* column of the Students table, ensuring that orders are only placed by valid students present in the Students table.

A third commonly used constraint is the NOT NULL constraint. It ensures that a column cannot have a NULL (empty) value. This is useful when certain information is mandatory. For example, in a Students table, the *name* column can be defined as NOT NULL so that every student record must include a name and cannot be left empty.

In conclusion, SQL constraints play a vital role in maintaining data integrity by enforcing rules such as uniqueness, valid relationships, and mandatory fields within a database

Question 3 : Explain the difference between LIMIT and OFFSET clauses in SQL. How would you use them together to retrieve the third page of results, assuming each page has 10 records?

Answer- In SQL, the LIMIT and OFFSET clauses are used to control the number of records returned in a query result, especially for pagination purposes. The LIMIT clause specifies the maximum number of records to be retrieved from a table, while the OFFSET clause specifies the number of records to skip before starting to return the results. In simple terms, LIMIT decides "how many rows to show," and OFFSET decides "from where to start showing."

For example, if we use SELECT * FROM Students LIMIT 10;, it will return only the first 10 records from the Students table. On the other hand, if we use SELECT * FROM Students LIMIT 10 OFFSET 10;, it will skip the first 10 records and return the next 10 records.

To retrieve the third page of results when each page contains 10 records, we need to skip the first 20 records (because page 1 has 10 and page 2 has 10). Therefore, we use LIMIT 10 OFFSET 20. The query will be:
SELECT * FROM Students LIMIT 10 OFFSET 20;

This will return records from position 21 to 30, which represent the third page of results.

In conclusion, LIMIT and OFFSET are commonly used together to implement pagination in SQL, allowing users to fetch data in smaller, manageable chunks

Question 4 : What is a Common Table Expression (CTE) in SQL, and what are its main benefits? Provide a simple SQL example demonstrating its usage.

Answer-A Common Table Expression (CTE) in SQL is a temporary result set that is defined using the WITH clause and can be referenced within a SELECT, INSERT, UPDATE, or DELETE query. It acts like a temporary table that exists only for the duration of the query. The main purpose of a CTE is to make complex queries easier to read, write, and maintain by breaking them into simpler and more understandable parts.

One of the main benefits of a CTE is improved readability, as it allows developers to structure queries in a logical way instead of writing long and complicated nested subqueries. Another advantage is reusability, since the CTE can be referenced multiple times within the same query. CTEs are also useful for performing recursive operations, such as working with hierarchical data like organizational structures or tree-like data

```
WITH AvgMarks AS (
   SELECT AVG(marks) AS avg_mark
   FROM Students
)
SELECT name, marks
FROM Students
WHERE marks > (SELECT avg_mark FROM AvgMarks);
```

In this example, the CTE named *AvgMarks* calculates the average marks of all students. This result is then used in the main query to select only those students whose marks are greater than the average. This approach makes the query more organized and easier to understand compared to writing everything in a single nested query.

In conclusion, a CTE is a powerful SQL feature that simplifies complex queries, improves readability, and allows better query structuring.

Question 5 : Describe the concept of SQL Normalization and its primary goals. Briefly explain the first three normal forms (1NF, 2NF, 3NF).

Answer- SQL Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. The main goal of normalization is to eliminate duplicate data, ensure data consistency, and make the database more efficient and easier to maintain. It involves dividing large tables into smaller, well-structured tables and defining relationships between them. By following normalization rules, we can avoid problems such as data anomalies during insertion, updating, or deletion.

The first normal form (1NF) ensures that a table contains only atomic (indivisible) values and that each column holds a single value. It also requires that each record can be uniquely identified. For example, a table should not store multiple phone numbers in a single column; instead, each phone number should be stored in a separate row.

The second normal form (2NF) builds on 1NF and ensures that all non-key attributes are fully dependent on the primary key. This means there should be no partial dependency. It is mainly applicable when a table has a composite primary key. For instance, if a table contains student_id and course_id as a combined primary key, then attributes like student_name should depend only on student_id and should be moved to a separate table.

The third normal form (3NF) ensures that there are no transitive dependencies, meaning non-key attributes should not depend on other non-key attributes. All attributes must depend only on the primary key. For example, if a table contains student_id, department_id, and department_name, then department_name depends on department_id, not directly on student_id. In such a case, department details should be stored in a separate table.

In conclusion, normalization helps in structuring databases efficiently by reducing redundancy and improving consistency, and the first three normal forms (1NF, 2NF, and 3NF) are the foundation for designing a well-organized relational database

Question 6 : Create a database named ECommerceDB and perform the following tasks

```
CREATE DATABASE ECommerceDB;
USE ECommerceDB;

-- 1. Create the following tables with appropriate data types and constraints:
CREATE TABLE Categories (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL UNIQUE,
    CategoryID INT,
    Price DECIMAL(10,2) NOT NULL,
    StockQuantity INT,
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);

CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    JoinDate DATE
);

CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE NOT NULL,
    TotalAmount DECIMAL(10,2),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

-- 2. Insert the following records into each table

INSERT INTO Categories VALUES
(1, 'Electronics'),
(2, 'Books'),
(3, 'Home Goods'),
(4, 'Apparel');

INSERT INTO Products VALUES
(101, 'Laptop Pro', 1, 1200.00, 50),
(102, 'SQL Handbook', 2, 45.50, 200),
(103, 'Smart Speaker', 1, 99.99, 150),
(104, 'Coffee Maker', 3, 75.00, 80),
(105, 'Novel: The Great SQL', 2, 25.00, 120),
(106, 'Wireless Earbuds', 1, 150.00, 100),
(107, 'Blender X', 3, 120.00, 60),
(108, 'T-Shirt Casual', 4, 20.00, 300);

INSERT INTO Customers VALUES
(1, 'Alice Wonderland', 'alice@example.com', '2023-01-10'),
(2, 'Bob the Builder', 'bob@example.com', '2022-11-25'),
```

```
(3, 'Charlie Chaplin', 'charlie@example.com', '2023-03-01'),
(4, 'Diana Prince', 'diana@example.com', '2021-04-26');

INSERT INTO Orders VALUES
(1001, 1, '2023-04-26', 1245.50),
(1002, 2, '2023-10-12', 99.99),
(1003, 1, '2023-07-01', 145.00),
(1004, 3, '2023-01-14', 150.00),
(1005, 2, '2023-09-24', 120.00),
(1006, 1, '2023-06-19', 20.00);

SELECT * FROM Categories;
SELECT * FROM Products;
SELECT * FROM Customers;
SELECT * FROM Orders;


##Question 7 : Generate a report showing CustomerName, Email, and the
-- TotalNumberofOrders for each customer. Include customers who have not
placed
-- any orders, in which case their TotalNumberofOrders should be 0. Order
the results
##by CustomerName.

-- Answer
SELECT
    c.CustomerName,
    c.Email,
    COALESCE(COUNT(o.OrderID), 0) AS TotalNumberOfOrders
FROM Customers c
LEFT JOIN Orders o
    ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.CustomerName, c.Email
ORDER BY c.CustomerName;


##Question 8 : Retrieve Product Information with Category: Write a SQL
query to
-- display the ProductName, Price, StockQuantity, and CategoryName for
all
-- products. Order the results by CategoryName and then ProductName
alphabetically.

-- Answer-
SELECT
    p.ProductName,
    p.Price,
    p.StockQuantity,
    c.CategoryName
FROM Products p
INNER JOIN Categories c
    ON p.CategoryID = c.CategoryID
ORDER BY c.CategoryName, p.ProductName;


## Question 9 : Write a SQL query that uses a Common Table Expression
(CTE) and a
-- Window Function (specifically ROW_NUMBER() or RANK()) to display the
```

```
-- CategoryName, ProductName, and Price for the top 2 most expensive
products in
-- each CategoryName

-- Answer-

WITH RankedProducts AS (
    SELECT
        c.CategoryName,
        p.ProductName,
        p.Price,
        ROW_NUMBER() OVER (
            PARTITION BY c.CategoryName
            ORDER BY p.Price DESC
        ) AS rank_num
    FROM Products p
    INNER JOIN Categories c
        ON p.CategoryID = c.CategoryID
)
SELECT
    CategoryName,
    ProductName,
    Price
FROM RankedProducts
WHERE rank_num <= 2
ORDER BY CategoryName, Price DESC;


## Question 10 : You are hired as a data analyst by Sakila Video Rentals,
a global movie
-- rental company. The management team is looking to improve decision-
making by
-- analyzing existing customer, rental, and inventory data.
-- Using the Sakila database, answer the following business questions to
support key strategic
-- initiatives.
-- Tasks & Questions:
-- 1. Identify the top 5 customers based on the total amount theyâ€™ve
spent. Include customer
-- name, email, and total amount spent.
-- 2. Which 3 movie categories have the highest rental counts? Display
the category name
-- and number of times movies from that category were rented.
-- 3. Calculate how many films are available at each store and how many
of those have
-- never been rented.
-- 4. Show the total revenue per month for the year 2023 to analyze
business seasonality.
-- 5. Identify customers who have rented more than 10 times in the last 6
months.

-- Answer-

use sakila ;

-- 1. Top 5 Customers by Total Spending
SELECT
    c.first_name,
```

```sql
    c.last_name,
    c.email,
    SUM(p.amount) AS total_spent
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email
ORDER BY total_spent DESC
LIMIT 5;

-- 2. Top 3 Movie Categories by Rental Count
SELECT
    cat.name AS category_name,
    COUNT(r.rental_id) AS rental_count
FROM category cat
JOIN film_category fc
    ON cat.category_id = fc.category_id
JOIN inventory i
    ON fc.film_id = i.film_id
JOIN rental r
    ON i.inventory_id = r.inventory_id
GROUP BY cat.name
ORDER BY rental_count DESC
LIMIT 3;

-- 3. Films Available at Each Store & Never Rented
SELECT
    s.store_id,
    COUNT(i.inventory_id) AS total_films,
    SUM(CASE WHEN r.rental_id IS NULL THEN 1 ELSE 0 END) AS never_rented
FROM store s
JOIN inventory i
    ON s.store_id = i.store_id
LEFT JOIN rental r
    ON i.inventory_id = r.inventory_id
GROUP BY s.store_id;

-- 4. Monthly Revenue for Year 2023
SELECT
    MONTH(payment_date) AS month,
    SUM(amount) AS total_revenue
FROM payment
WHERE YEAR(payment_date) = 2023
GROUP BY MONTH(payment_date)
ORDER BY month;


-- 5. Customers with More Than 10 Rentals (Last 6 Months)
SELECT
    c.first_name,
    c.last_name,
    c.email,
    COUNT(r.rental_id) AS total_rentals
FROM customer c
JOIN rental r
    ON c.customer_id = r.customer_id
WHERE r.rental_date >= '2005-01-01'
GROUP BY c.customer_id, c.first_name, c.last_name, c.email
```

```
HAVING COUNT(r.rental_id) > 10;
```